SEDS519 HW1 Report
Petition Management System

Student Name
323011014

April 14, 2026

1 Introduction

This report explains the design and implementation of the Petition Management System developed for SEDS519 HW1. The application is a desktop GUI program that allows users to select built-in or user-defined petition templates, create petitions from scratch, edit petition details, save them as reusable templates, save them as drafts or register them, and browse saved petitions with filtering support.

The homework specifically requires the use of the following design patterns:

- Factory Method
- Singleton
- Iterator
- Prototype

The implementation satisfies these requirements through a petition creation layer, a central petition registry, a filtered petition iterator, and cloning support for template-based editing. The final version also includes a separate template persistence layer so that users can define their own templates from scratch and reuse them later.

2 Project Overview

The system is organized into the following modules:

- `models`: petition domain classes
- `factories`: creation logic for petition subtypes
- `templates`: built-in template catalog
- `registry`: petition and template persistence
- `iterators`: traversal and filtering logic
- `gui`: Flet-based desktop user interface
- `utils`: validation helpers

The application starts from `src/main.py`, launches the Flet GUI, loads template petitions, loads previously saved petitions from JSON storage, and allows the user to create editable petition copies from built-in templates, user-defined templates, or blank petition types.

3 Factory Method Pattern

3.1 Intent

The Factory Method pattern is used to create petitions of different types without forcing the client code to directly instantiate concrete petition classes.

3.2 Implementation

The abstract base factory is `PetitionFactory`. It declares the `create_petition(...)` method and acts as a shared creation interface for all petition factories.

Listing 1: Abstract petition factory

```
class PetitionFactory(ABC):
    @abstractmethod
    def create_petition(
        self,
        title: str,
        body: str,
        petitioner: str,
        created_by: str,
        status: str = "draft",
        attachment_required: bool = False,
        attachments: list[str] | None = None,
    ) -> Petition:
        raise NotImplementedError
```

Two concrete factories implement this interface:

- `AcademicPetitionFactory`
- `AdministrativePetitionFactory`

Each concrete factory returns a different petition subtype with its own default receiver and type metadata:

- academic petitions default to `receiver = "Dean"`
- administrative petitions default to `receiver = "Administrative Office"`
- factory calls can also specify whether attachments are required

3.3 How it is used

The template catalog uses these factories to create built-in petition templates. For example, the academic templates are generated by creating an `AcademicPetitionFactory` instance and repeatedly calling `create_petition(...)`. The internship approval template is created with `attachment_required=True`, which demonstrates that different petition instances can carry different rules at creation time.

This design keeps template creation logic centralized and prevents the GUI from being tightly coupled to concrete petition classes.

3.4 Why this is appropriate

Factory Method is appropriate here because the application creates more than one petition subtype and each subtype may evolve with different defaults, receivers, or future attachment rules. The pattern makes these differences explicit while keeping creation logic clean and extensible.

4 Singleton Pattern

4.1 Intent

The Singleton pattern is used to ensure that the petition registry is shared across the entire application.

4.2 Implementation

The `PetitionRegistry` class stores a class-level variable named `_instance`. Its `__new__` method checks whether an instance already exists:

- if not, it creates one object
- if yes, it returns the existing object

Listing 2: Singleton registry creation logic

```
class PetitionRegistry:
    _instance: "PetitionRegistry | None" = None

    def __new__(cls) -> "PetitionRegistry":
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._petitions = []
            cls._instance._storage_dir = (
                Path(__file__).resolve().parents[2] / "data" / "
                    petitions"
            )
            cls._instance._storage_dir.mkdir(parents=True, exist_ok=
                True)
            cls._instance.load_all_petitions()
        return cls._instance
```

4.3 Responsibilities of the registry

The registry acts as the single control structure for petitions in the system. It supports:

- adding a petition
- registering a petition
- retrieving all petitions
- retrieving only drafts
- retrieving only registered petitions
- loading petitions from JSON storage
- saving petitions back to JSON storage

4.4 Why this is appropriate

The homework requires a single shared registry over petitions that are either in draft state or already registered. Singleton fits this requirement directly because every part of the application works with the same in-memory petition collection and the same storage directory. A separate `TemplateRegistry` is used for user-defined reusable templates, but the Singleton requirement is specifically fulfilled by `PetitionRegistry`.

5 Iterator Pattern

5.1 Intent

The Iterator pattern is used to traverse petitions stored in the registry while optionally filtering them by type or status.

5.2 Implementation

The `PetitionIterator` class accepts:

- a petition list
- an optional petition type filter
- an optional status filter

It implements both `__iter__` and `__next__`, so it can be used in normal Python iteration constructs.

Listing 3: Filtered petition iterator

```
class PetitionIterator:
    def __iter__(self) -> "PetitionIterator":
        return self

    def __next__(self) -> Petition:
        while self._index < len(self._petitions):
            petition = self._petitions[self._index]
            self._index += 1
            matches_type = self._petition_type is None or getattr(
                petition, "petition_type", None
            ) == self._petition_type
            matches_status = self._status is None or petition.status ==
                self._status
            if matches_type and matches_status:
                return petition
        raise StopIteration
```

5.3 How it is used

The GUI uses `PetitionIterator` when displaying saved petitions. Instead of placing filtering logic directly inside the UI, the iterator handles type-agnostic traversal and optional filtering such as:

- academic petitions only
- administrative petitions only
- draft petitions only
- registered petitions only

5.4 Why this is appropriate

This matches the homework requirement closely: the registry stores the petitions and the iterator is responsible for traversing them while supporting filtered views.

6 Prototype Pattern

6.1 Intent

The Prototype pattern is used to create a new editable petition by copying an existing petition template.

6.2 Implementation

The base `Petition` model defines a `clone()` method that returns a deep copy:

Listing 4: Prototype cloning support

```
@dataclass
class Petition:
    ...
    def clone(self) -> "Petition":
        return deepcopy(self)
```

6.3 How it is used

The built-in petition templates are created once through the factory layer. User-defined templates are stored separately and loaded from template storage. When the user selects any template in the GUI, the selected template is cloned and the clone becomes the editable petition instance. This ensures that:

- the original template remains unchanged
- every user works on a separate petition object
- copying works for both academic and administrative petition types
- the same template can be reused many times

6.4 Why this is appropriate

Prototype is well-suited to template-based petition creation. The user starts from an existing petition shape, copies it, and customizes the copy instead of constructing a new object from zero.

7 Validation Logic

The homework asks for validity checking, especially for empty petition bodies and missing attachments when attachments are required.

This logic is implemented in `utils/validators.py`:

Listing 5: Petition validity rules

```
def is_valid_petition(petition: Petition) -> bool:
    if not petition.body.strip():
        return False
    if petition.attachment_required and not petition.attachments:
        return False
    return True
```

This helper is used by the GUI before a petition is saved or registered. As a result, invalid petitions are rejected before they enter persistent storage. In the final version, the `Internship Approval Request` template is configured to require attachments, so the validation rule is now demonstrated by an actual petition flow rather than only existing as a helper.

8 GUI Design

The application includes a desktop GUI built with the Flet framework. The GUI supports the main user tasks required by the assignment:

- browsing built-in petition templates
- browsing user-defined templates
- creating a blank academic or administrative petition from scratch
- selecting a template for editing
- editing title, body, petitioner, receiver, and creator fields
- saving the current editor state as a reusable template
- attaching files
- saving a petition as draft
- registering a petition
- browsing already saved petitions
- filtering saved petitions by type and status

The GUI is event-driven. Button clicks and dropdown changes trigger handler functions that delegate work to the registry, validation helper, and iterator. This keeps the interface responsive while separating display logic from core petition logic.

9 Persistence

Petitions are persisted as JSON files under `data/petitions`. The registry is responsible for all persistence:

- `save_all_petitions()` serializes petitions to JSON
- `load_all_petitions()` reconstructs petition objects on startup

The registry also restores the correct petition subtype from stored data. If `petition_type` is academic, it rebuilds an `AcademicPetition`; if it is administrative, it rebuilds an `AdministrativePetition`. Otherwise, it falls back to the base `Petition` class.

Reusable user-defined templates are persisted separately under `data/templates`. This logic is handled by `TemplateRegistry`, which mirrors the save/load responsibilities of the petition registry without being part of the Singleton requirement.

10 UML Class Diagram

The project includes a UML class diagram source in the `docs/uml` folder. The source was updated to reflect:

- the new `TemplateRegistry`
- the revised factory method signature with `attachment_required`
- the GUI support for starting petitions from scratch and saving custom templates

Figure 1 presents the class diagram used to describe the class structure and interactions.

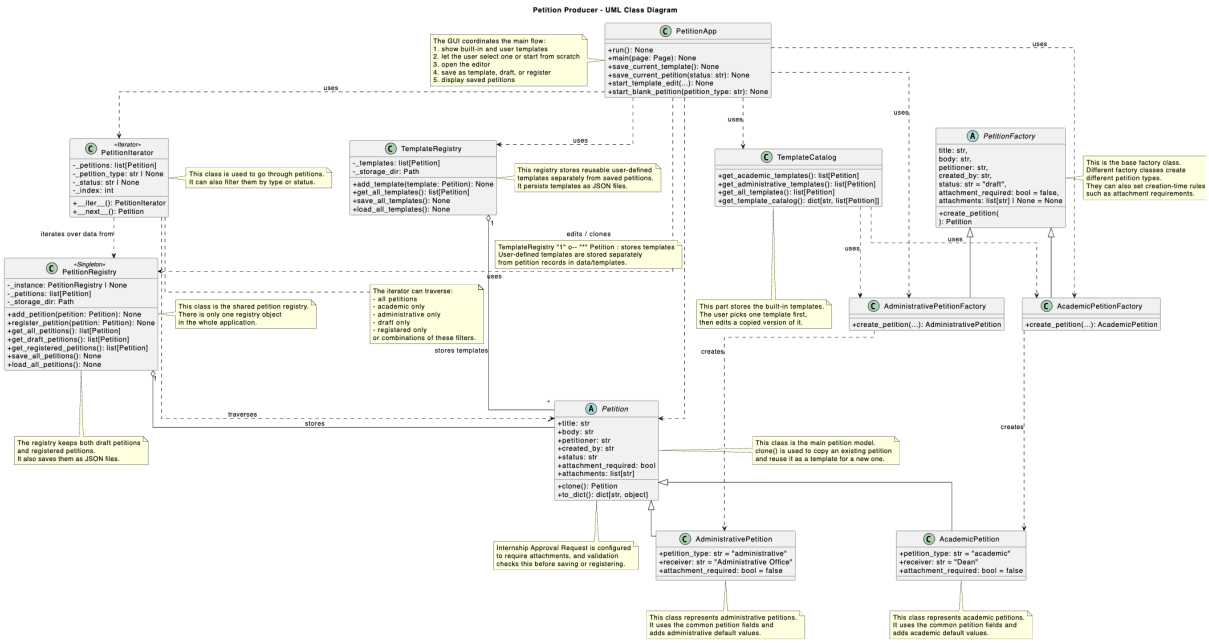

Figure 1: UML class diagram of the Petition Management System

11 Conclusion

The Petition Management System demonstrates the required design patterns in a coherent application rather than as isolated examples. Factory Method is used to create different petition subtypes, Singleton centralizes petition storage, Iterator provides filtered traversal over saved petitions, and Prototype enables template cloning for new petition creation. The system also includes validation, JSON persistence for both petitions and templates, a usable GUI, and UML documentation.

Overall, the design keeps responsibilities separated and makes the system easier to extend. New petition types, additional validation rules, or new registry operations can be added without major restructuring of the current codebase.